

数据结构与算法

第2章 线性表

张丽淼

光电信息获取与防护技术全国重点实验室

zhanglm@ahu.edu.cn

2026年3月18日

上节回顾

1、顺序表、链表的优缺点

顺序表：存储密度大、随机存取

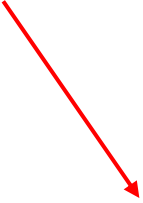
操作复杂度高、浪费储存空间

链表：增删操作效率高、空间利用率高

储存密度低、顺序存取

回顾：单链表的存储结构定义

```
typedef struct LNode{  
    ElemType data;    //数据域  
    struct LNode *next; //指针域  
}LinkNode; // 单链表结点类型的别名
```



类型名，不是变量名

实现——结构体

data	next
------	------

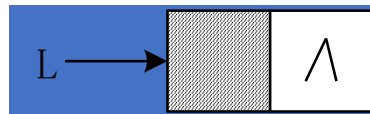
2.5.2 单链表基本操作的实现



1. 初始化、判断空表、销毁、清空、求表长
2. 取值
3. 查找
4. 插入
5. 删除

单链表基本操作的算法实现

- 初始化 (构造一个空表)



【算法思路】

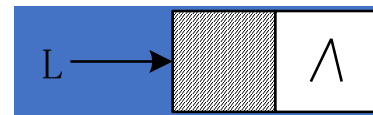
- (1) 生成新结点作头结点，用头指针L指向头结点。
- (2) 头结点的指针域置空。

【算法描述】

```
void InitList (LinkNode *&L){ //构造一个空的链表L
    L=new LinkNode; // L指针头结点, C++
    // L=(LinkNode *)malloc(sizeof(LinkNode)); //C语言
    L->next=NULL; //头结点的指针域为空
}
```

单链表基本操作的算法实现

- 判断表是否为空



空表：链表中无元素，称为空链表（头指针和头结点仍然在）

【算法思路】 判断头结点的指针域是否为空

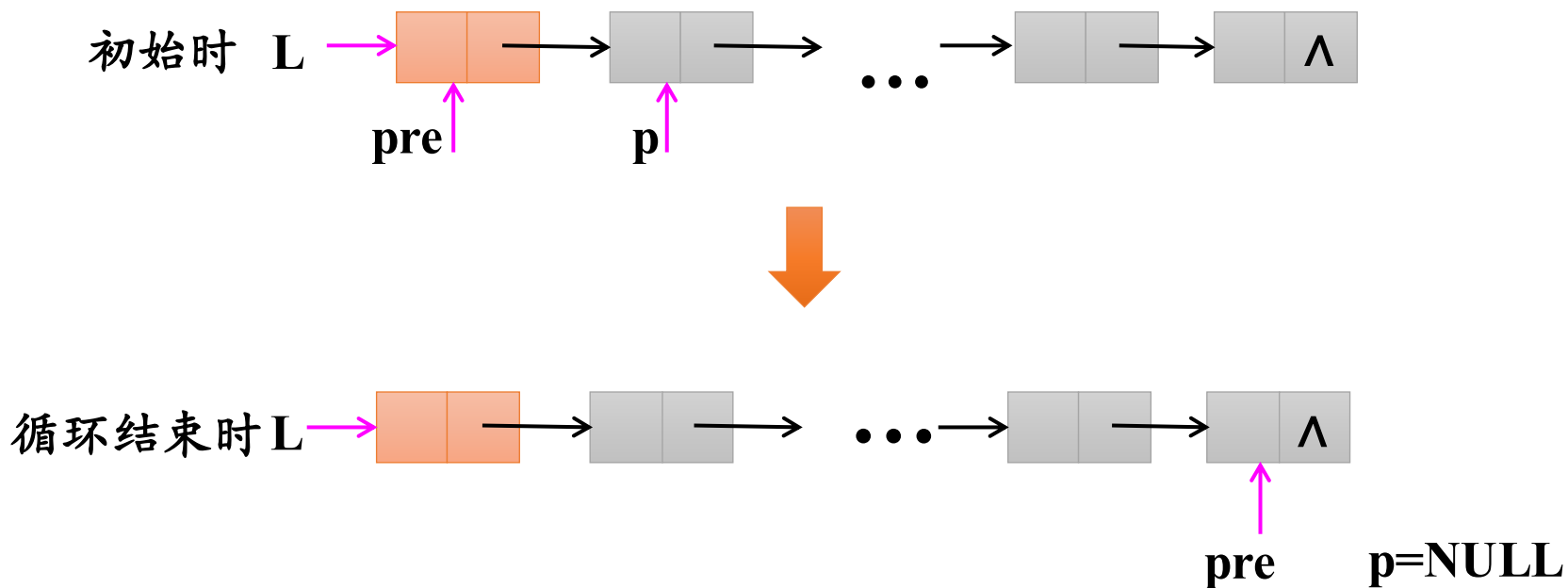
```
bool ListEmpty (LinkNode *L){  
    //若L为空表，则返回1，否则返回0  
    return (L->next==NULL)  
}
```

单链表基本操作的算法实现

- 销毁：将包括头结点在内的所有节点释放掉
【算法思路】从头指针开始，依次释放所有结点

```
void DestroyList (LinkNode *&L)
{
    LinkNode *pre=L, *p=L->next; //pre指向p的前驱结点
    while (p!=NULL) //扫描单链表L
    { free(pre); //释放pre结点，也可以delete pre
      pre=p; //pre、p同步后移一个结点
      p=pre->next;
    }
    free(pre); //循环结束时，p为NULL，pre指向尾结点，释放它
}
```

双指针法:



销毁链表后，就不能对链表进行任何操作，否则为访问权限冲突

单链表基本操作的算法实现

- 清空：将所有除头节点以外的存放有数据的节点释放，成为空链表，头结点和头指针仍存在。

【算法思路】依次释放所有节点，并将头结点指向NULL

```
void ClearList (LinkNode *&L){
```

```
    // 将L重置为空表
```

```
    LinkNode *p, *q;
```

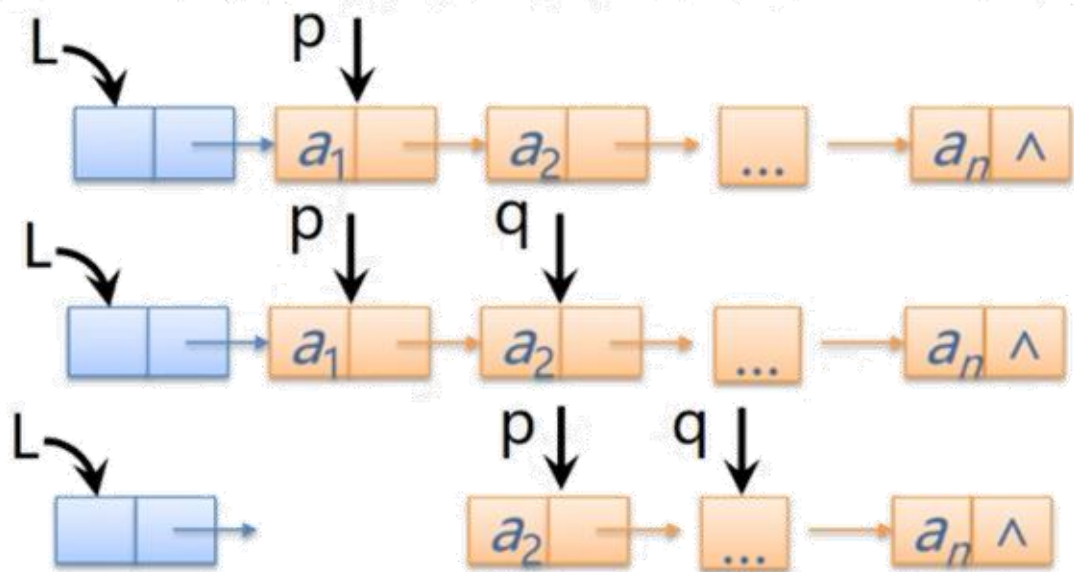
```
    p=L->next; //p指向第一个结点/首元结点
```

```
    while(p)    //没到表尾
```

```
        { q=p->next; free(p);    p=q; } //q保存下个结点地址
```

```
    L->next=NULL; //头结点指针域为空
```

```
}
```



$p = L \rightarrow \text{next}$

$q = p \rightarrow \text{next}$
 $\text{free}(p)$

$p = q$
 $q = p \rightarrow \text{next}$

清空链表后，链表还是可以使用

单链表基本操作的算法实现

“数”结点：

- 指针p依次指向各个结点
- 从第一个元素开始“数”
- 一直“数”到最后一个结点

• 求表长

```
int ListLength (LinkNode *L)
```

```
{
```

```
    int n=0;
```

```
    LinkNode *p=L;    //p指向头结点，n置为0（即头  
结点的序号为0）
```

```
    while (p->next!=NULL)
```

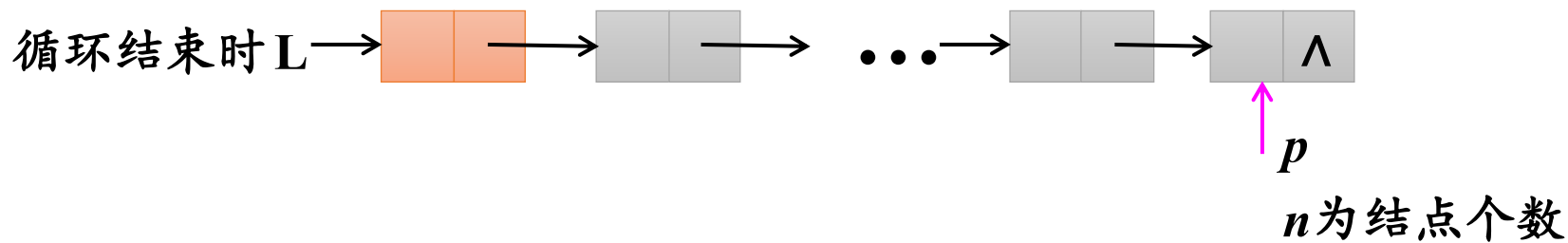
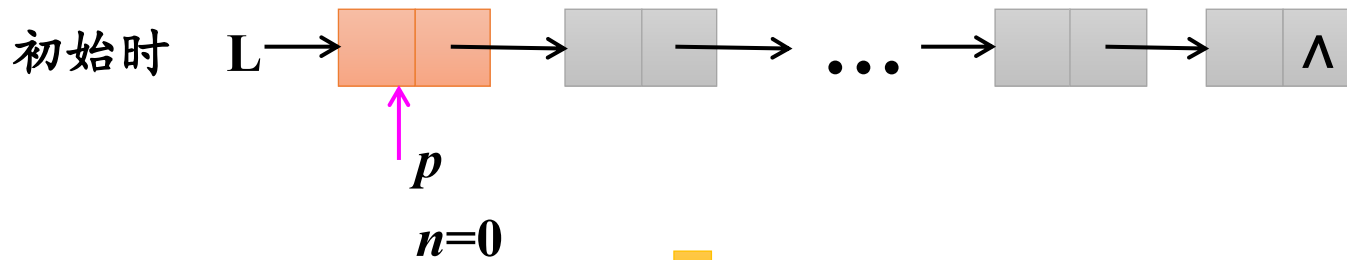
```
    {    n++;
```

```
        p=p->next;
```

```
    }
```

```
    return(n);    //循环结束，p指向尾结点，其序号n  
为结点个数
```

```
}
```



注意头结点不计入长度。

2.5.2 单链表基本操作的实现

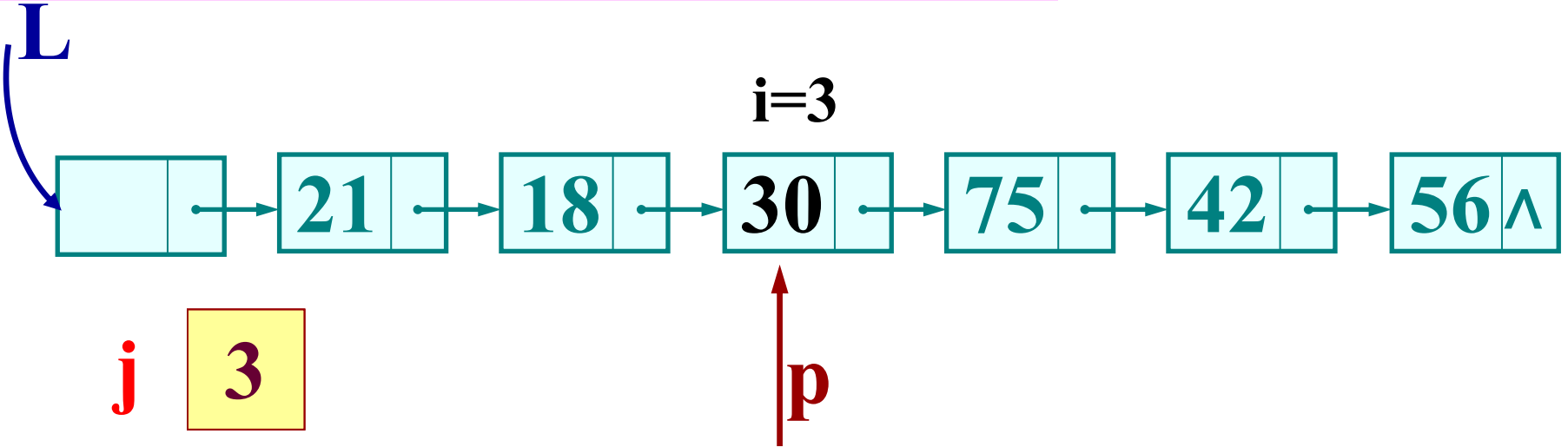


1. 初始化、判断空表、销毁、清空、求表长
2. 取值
3. 查找
4. 插入
5. 删除

2. 取值 (根据位置*i*获取相应位置数据元素的内容)

- 思考：顺序表里如何找到第*i*个元素？
- 链表不是随机存取结构
- 链表的查找：要从链表的头指针出发，顺着链域next逐个结点往下搜索，直至搜索到第*i*个结点为止。

例：取出表中 $i=3$ 的元素



【算法步骤】

- ✓ 从头结点 ($j=0$) 顺链扫描，用指针 p 指向当前扫描到的结点。（也可从首元结点开始扫描）
- ✓ j 做计数器，累计当前扫描过的结点数， j 初值为1。
- ✓ 当 p 指向扫描到的下一结点时，计数器 j 加1。
- ✓ 当 $j=i$ 时， p 所指的结点就是要找的第 i 个结点。

2. 取值 (根据位置*i*获取相应位置数据元素的内容)

//获取线性表L中的某个数据元素的内容

```
bool GetElem(LinkNode *L, int i, ElemType &e)
```

```
{
```

```
    int j=0;
```

```
    LinkNode *p=L;           //p指向头结点, j置为0 (即头结点的序号为0)
```

```
    while (j<i && p!=NULL)
```

```
    { j++;
```

```
      p=p->next;
```

```
    }
```

```
    if (p==NULL) //不存在第i个数据结点, 返回false
```

```
        return false;
```

```
    else
```

```
        //存在第i个数据结点, 返回true
```

```
    { e=p->data;
```

```
      return true;
```

```
    }
```

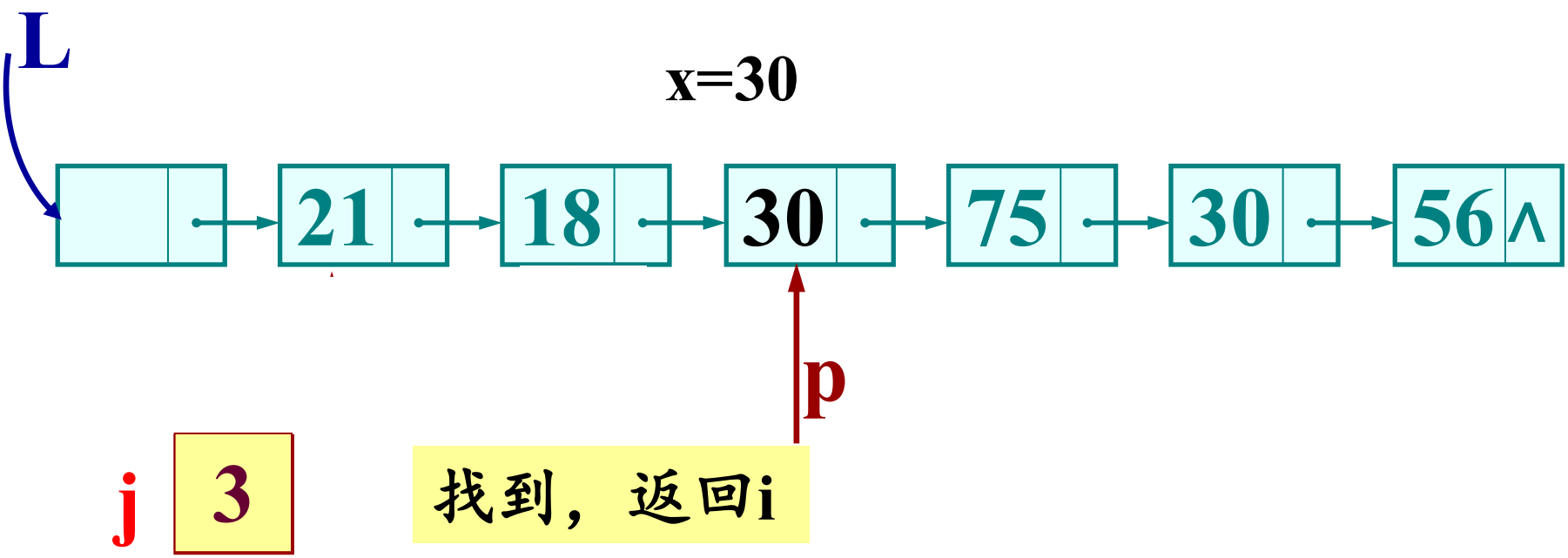
```
}
```

2.5.2 单链表基本操作的实现



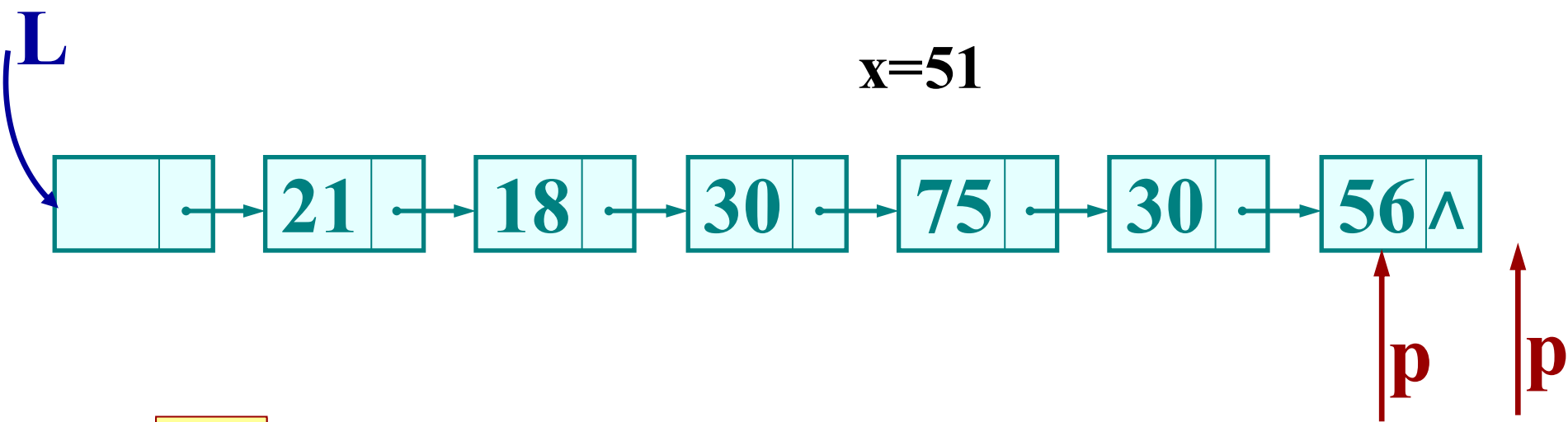
1. 初始化、判断空表、销毁、清空、求表长
2. 取值 $O(n)$
3. 查找
4. 插入
5. 删除

3.查找 (根据指定数据获取数据所在的位置)



- ✓从第一个结点起，依次和e相比较。
- ✓如果找到一个其值与e相等的数据元素，则返回其在链表中的“位置”或地址；
- ✓如果查遍整个链表都没有找到其值和e相等的元素，则返回0或“NULL”。

3.查找 (根据指定数据获取数据所在的位置)



j **7** 未找到，返回0

- ✓从第一个结点起，依次和e相比较。
- ✓如果找到一个其值与e相等的数据元素，则返回其在链表中的“位置”或地址；
- ✓如果查遍整个链表都没有找到其值和e相等的元素，则返回0或“NULL”。

【算法描述1】

//在线性表L中查找值为e的数据元素

```
int LocateElem(LinkNode *L, ElemType e)
```

```
{
```

```
    int i=1;
```

```
    LinkNode *p=L->next;    //p指向首元结点，i置为1
```

```
    while (p!=NULL && p->data!=e)
```

```
    { p=p->next;                //查找data值为e的结点，其序号为i
```

```
        i++;
```

```
    }
```

```
    if (p==NULL)                //不存在元素值为e的结点，返回0
```

```
        return 0;
```

```
    else                        //存在元素值为e的结点，返回其逻辑序号i
```

```
        return i;
```

```
}
```

算法的时间复杂度为 $O(n) \Rightarrow$ 不具有随机存取特性

思考：

(1) 在刚才的操作中，指针变量p的后移操作能否通过p++来完成？为什么？

(2) 什么情况下，一个指针变量p作这样的p++是有意义的？

思考：

(1) 在刚才的操作中，指针变量p的后移操作能否通过p++来完成？为什么？

不能。因为链表中的结点在内存中不是连续存储的。p++是指针的算术运算，会使p指向当前地址之后的下一内存单元，但这并不等于链表中的下一个结点，会导致访问非法内存或错误数据。

(2) 什么情况下，一个指针变量p作这样的p++是有意义的？

思考：

(1) 在刚才的操作中，指针变量p的后移操作能否通过p++来完成？为什么？

不能。因为链表中的结点在内存中不是连续存储的。p++是指针的算术运算，会使p指向当前地址之后的下一内存单元，但这并不等于链表中的下一个结点，会导致访问非法内存或错误数据。

(2) 什么情况下，一个指针变量p作这样的p++是有意义的？

当**指针指向连续存储的数组元素**（如顺序表、静态数组、动态分配的连续内存块）时，p++才有意义，可以使指针指向下一个元素。例如遍历数组时，p++可以依次访问每个元素。

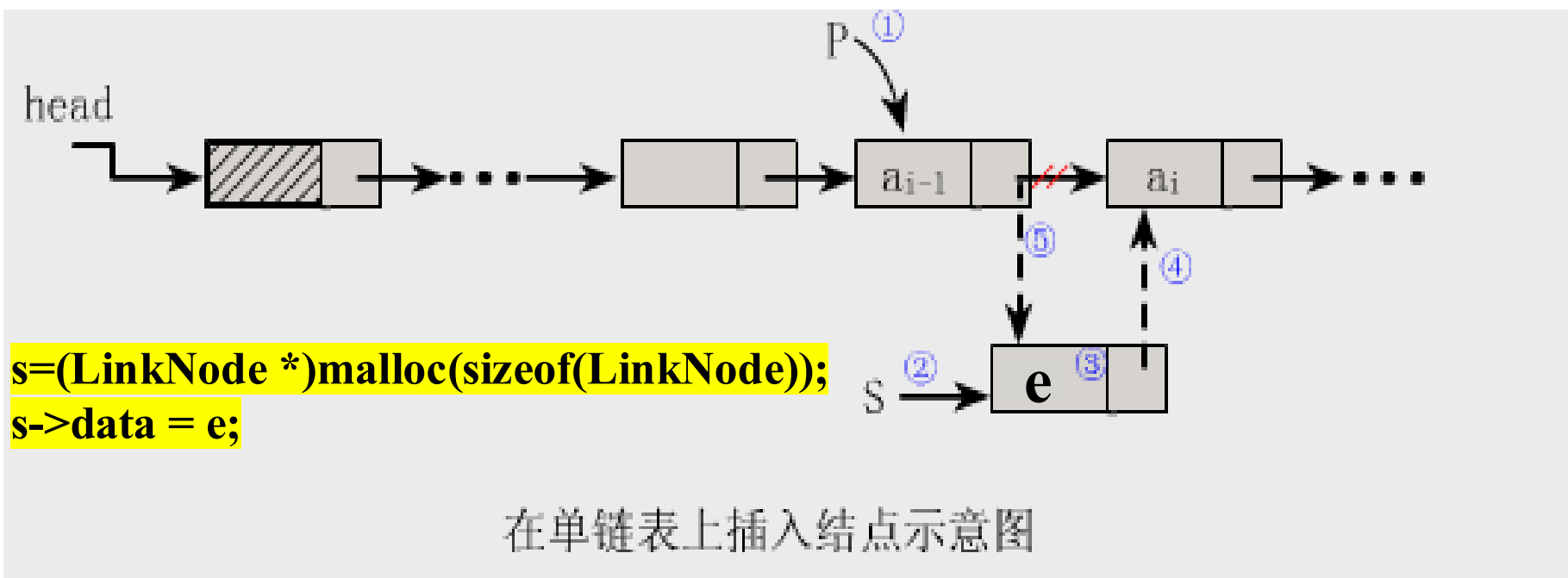
2.5.2 单链表基本操作的实现



1. 初始化、判断空表、销毁、清空、求表长
2. 取值 $O(n)$
3. 查找 $O(n)$
4. 插入
5. 删除

4. 插入 (在第 i 个结点之前插入值为 e 的新结点)

- 将值为 e 的新结点插入到表的第 i 个结点的位置上, 即插入到 a_{i-1} 与 a_i 之间

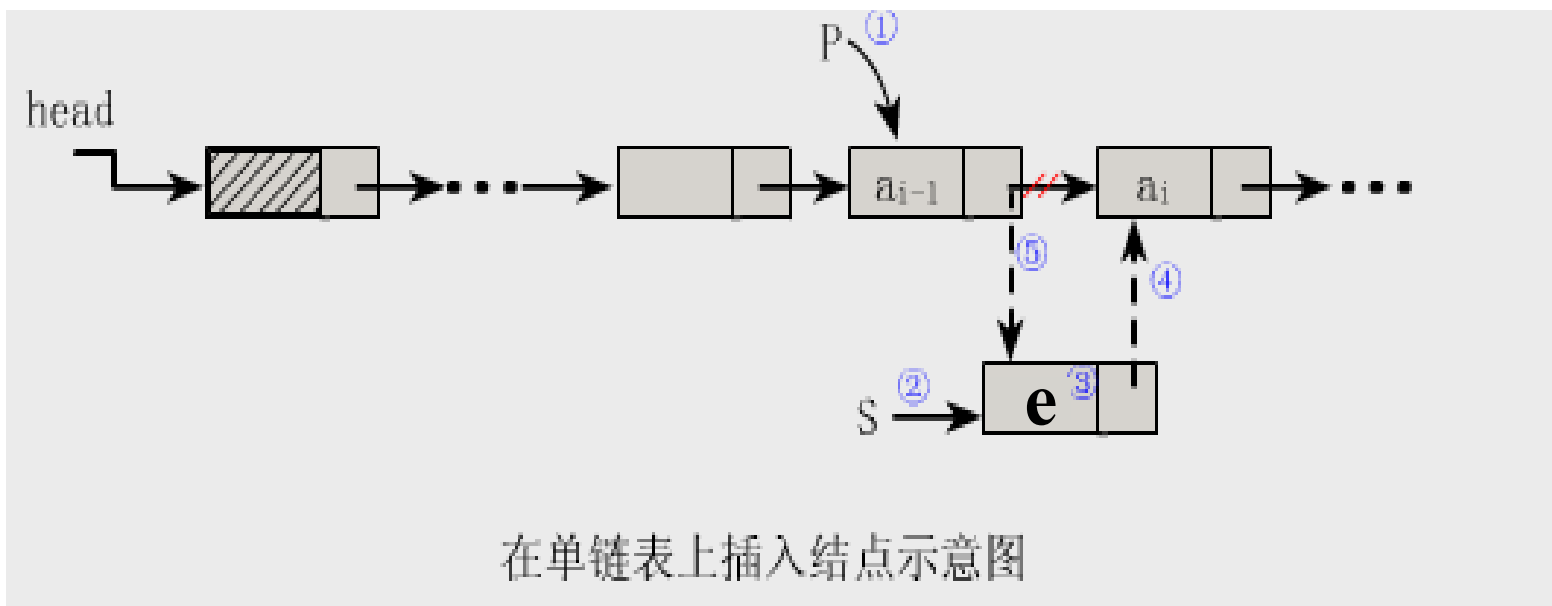


$s \rightarrow \text{next} = p \rightarrow \text{next}; \quad p \rightarrow \text{next} = s$

思考: 上述两步骤能互换么?
不可以, 会丢失 a_i 的地址!

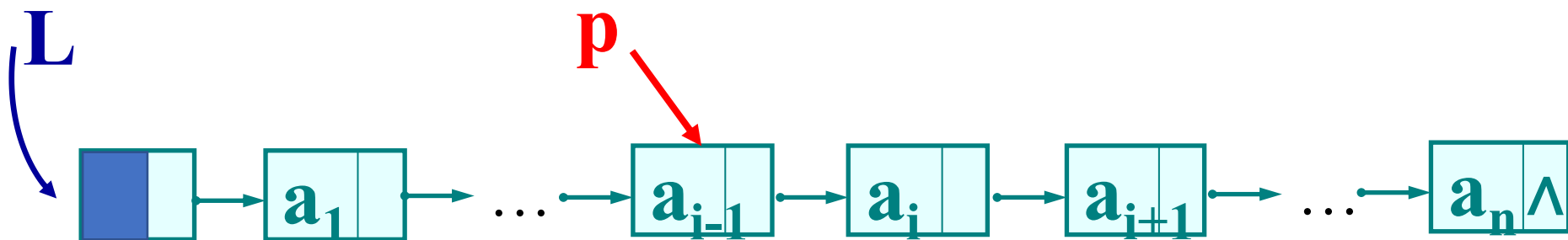
【算法步骤】

- (1) 找到 a_{i-1} 存储位置 p
- (2) 生成一个新结点 $*s$
- (3) 将新结点 $*s$ 的数据域置为 e
- (4) 新结点 $*s$ 的指针域指向结点 a_i
- (5) 令结点 $*p$ 的指针域指向新结点 $*s$



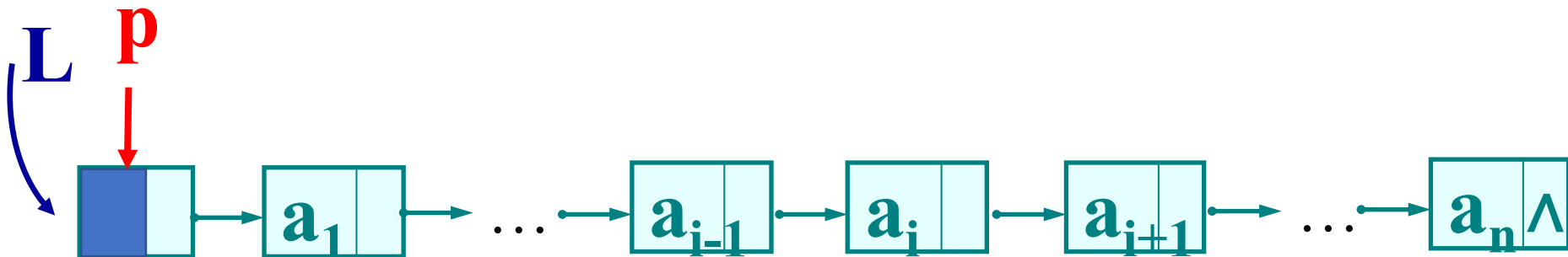
思考：

如何找到 a_{i-1} 存储位置 p , 即如何将指针 p 指向第 $i-1$ 个元素？



思考：

如何找到 a_{i-1} 存储位置 p , 即如何将指针 p 指向第 $i-1$ 个元素?


$$*p = L;$$

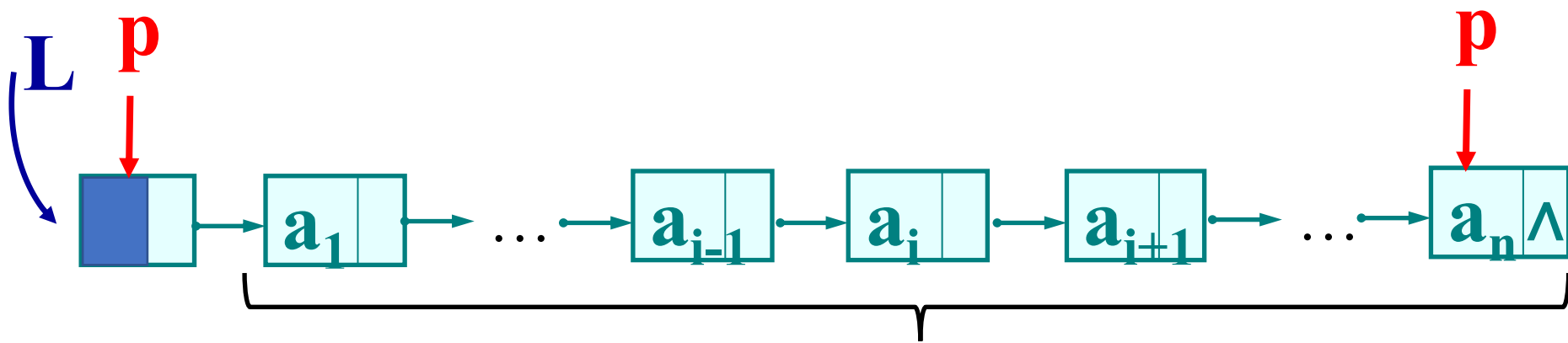
初始化p为什么指向头结点而不是首元结点？

```
while(
```

p=p->next;

思考：

如何找到 a_{i-1} 存储位置 p , 即如何将指针 p 指向第 $i-1$ 个元素？



$*p = L; j = 0;$

i 的取值范围: $[1, n+1]$

while($p \neq \text{NULL} \ \&\& \ j < i-1$)

{ $p = p \rightarrow \text{next}; j++;$ }

引入计数器 j , 保证与 p 的同步, 初值为0; 随着 p 移动, $j++$, 很明显
 $j < i-1$

【算法描述】

//在L中第i个元素之前插入数据元素e

bool ListInsert(LinkNode *&L, int i, ElemType e)

{ int j=0;

LinkNode *p=L, *s; //p指向头结点, j置为0

while (j<i-1 && p!=NULL). //或while(p&& j<i-1)

{ j++;

p=p->next;

}

if (p==NULL) //未找到第i-1个结点, 返回false

return false;

else //找到第i-1个结点p, 插入并返回true

{

s=(LinkNode *)malloc(sizeof(LinkNode)); //或C++中 s=new LinkNode;

s->data=e; //创建新结点s, 其data域置为e

s->next=p->next; //将s插入到p之后

p->next=s;

return true;

}

}

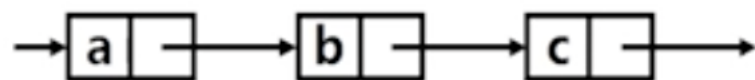
2.5.2 单链表基本操作的实现



1. 初始化、判断空表、销毁、清空、求表长
2. 取值 $O(n)$
3. 查找 $O(n)$
4. 插入 $O(n)$
5. 删除

5. 删除 (删除第 i 个结点)

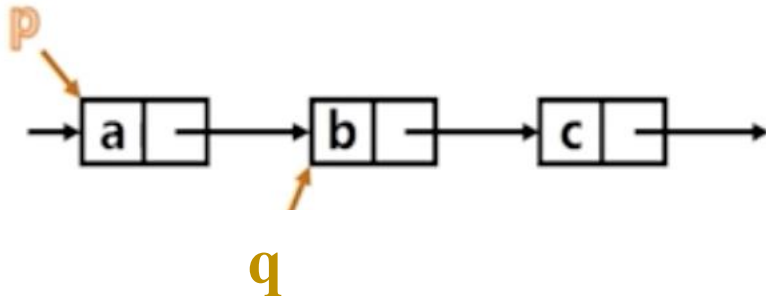
- 将表的第 i 个结点删去



先在单链表 L 中找到第 $i-1$ 个结点 p ，若存在这样的结点，且也存在后继结点，则删除该后继结点。

5. 删除 (删除第 i 个结点)

- 将表的第 i 个结点删去, 先找 i-1

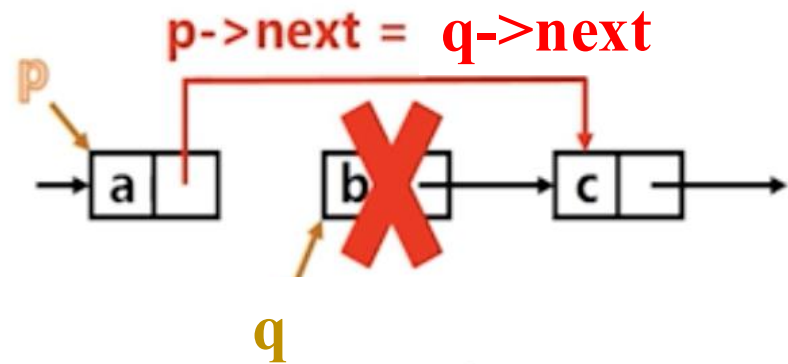


`q = p->next ;`

`p->next = q->next;`

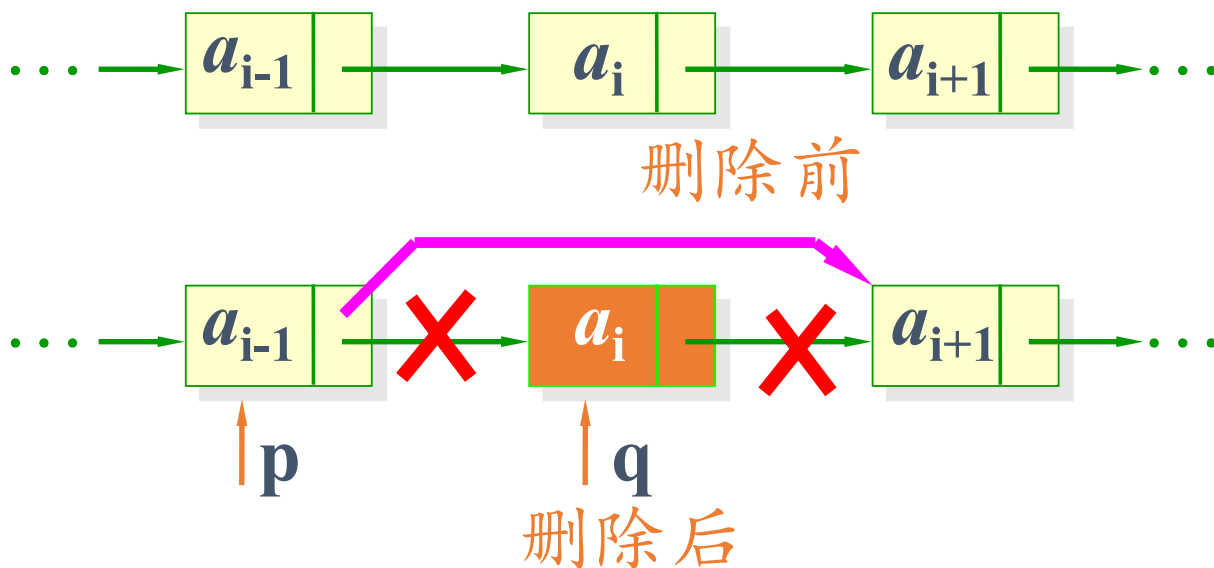
`e = q->data; //要返回被删元素的值, 要先保存data再释放`

`free (q);`



【算法步骤】

- (1) 找到 a_{i-1} 存储位置 p
- (2) 临时保存结点 a_i 的地址在 q 中，以备释放
- (3) 令 $p \rightarrow \text{next}$ 指向 a_i 的直接后继结点
- (4) 将 a_i 的值保留在 e 中
- (5) 释放 a_i 的空间



【算法描述】

//将线性表L中第i个数据元素删除

bool ListDelete(LinkNode *&L, int i, ElemType &e)

{ int j=0;

LinkNode *p=L, *q; //p指向头结点, 头结点序号j置为0

while (j<i-1 && p!=NULL) //查找第i-1个结点

{ j++;

p=p->next;

}

if (p==NULL) //未找到第i-1个结点, 返回false

return false;

else //找到第i-1个结点p

{ q=p->next; //q指向第i个结点

if (q==NULL) //若不存在第i个结点, 返回false

return false;

e=q->data;

p->next=q->next; //从单链表中删除q结点

free(q); //释放q结点

return true; //返回true表示成功删除第i个结点

}

}

2.5.2 单链表基本操作的实现



1. 初始化、判断空表、销毁、清空、求表长
2. 取值 $O(n)$
3. 查找 $O(n)$
4. 插入 $O(n)$
5. 删除 $O(n)$

链表的运算时间效率分析

1. **查找**: 因线性链表只能顺序存取, 即在查找时要从头指针找起, 查找的时间复杂度为 $O(n)$ 。
2. **插入和删除**:
 - 在单链表中尾部或者任意位置进行前插或删除操作, 由于要从头查找前驱结点, 所耗时间复杂度为 $O(n)$ 。
 - **特殊**: 在头部插入一个元素, 因线性链表不需要移动元素, 只要修改指针, 一般情况下时间复杂度为 $O(1)$ 。

2.5.2 单链表基本操作的实现



1. 初始化、判断空表、销毁、清空、求表长
2. 取值 $O(n)$
3. 查找 $O(n)$
4. 插入 $O(n)$
5. 删除 $O(n)$
6. 单链表的建立

单链表的建立

- 从一个空表（初始化）出发，不断往其插入元素的过程。

- 链表的创建方法

- 头插法

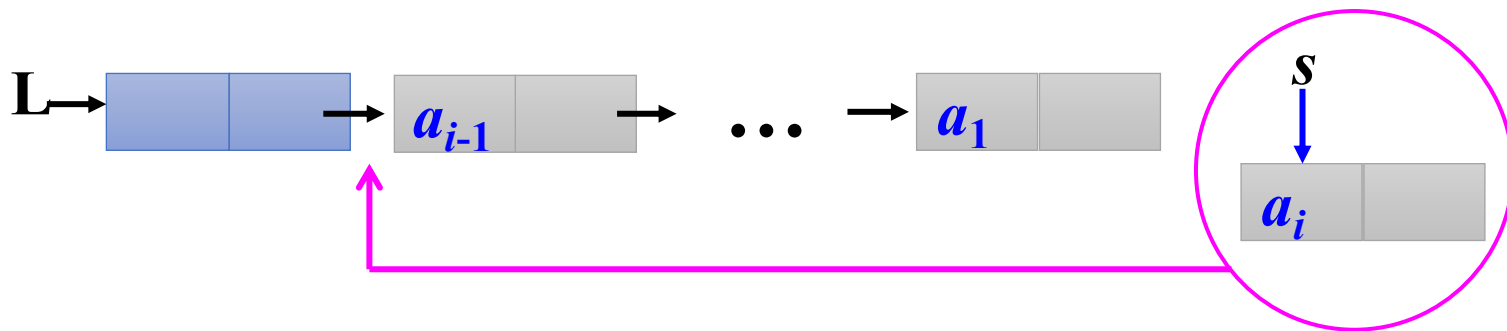
每次都将在新结点作为链表的第一个结点而插入，即始终插入到链表的第一个结点前

- 尾插法

每次都将在新结点作为链表的尾结点而插入，即始终插入到链表的最后一个结点之后

单链表的建立—— (1) 头插法建表

- 从一个空表开始，创建一个头结点。
- 依次读取字符数组 a 中的元素，生成新结点
- 将新结点插入到当前链表的表头上，直到结束为止。



注意：链表的结点顺序与逻辑次序相反。

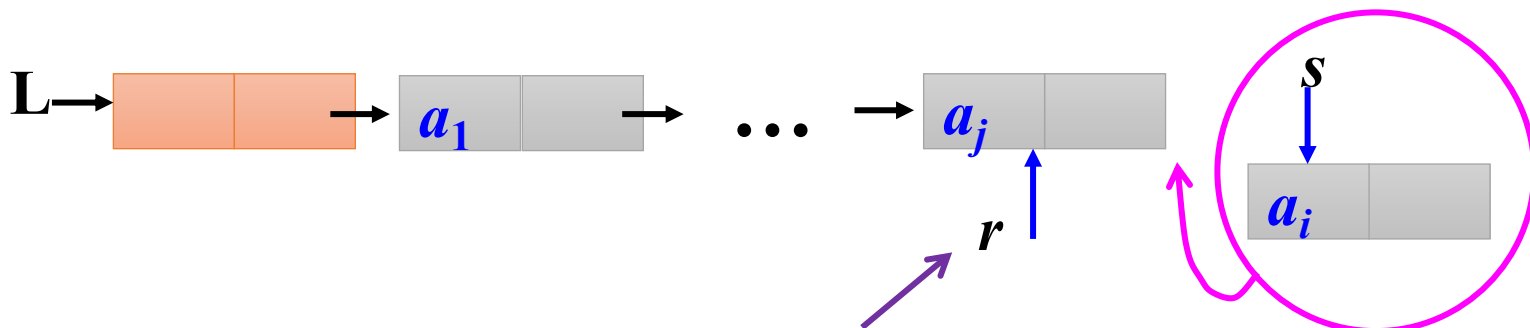
头插法建表算法如下：

```
void CreateListF(LinkNode *&L, ElemType a[], int n)
{ LinkNode *s;
  int i;
  L=(LinkNode *)malloc(sizeof(LinkNode));
  L->next=NULL;  //创建头结点，其next域置为NULL
  for (i=0;i<n;i++)  //循环建立数据结点
  { s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=a[i];  //创建数据结点s
    s->next=L->next; //将s插在原开始结点之前，头结点之后
    L->next=s;
  }
}
```

O(n)

单链表的建立—— (2) 尾插法建表

- 从一个空表开始，创建一个头结点。
- 依次读取字符数组 a 中的元素，生成新结点
- 将新结点插入到当前链表的表尾上，直到结束为止。



增加一个尾指针 r ，使其始终指向当前链表的尾结点

注意：链表的结点顺序与逻辑次序相同。

尾插法建表算法如下:

```
void CreateListR(LinkNode *&L, ElemType a[], int n)
{ LinkNode *s, *r;
  int i;
  L=(LinkNode *)malloc(sizeof(LinkNode)); //创建头结点
  r=L;                                     //r始终指向尾结点, 开始时指向头结点
  for (i=0;i<n;i++)                       //循环建立数据结点
  {
    s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=a[i];                          //创建数据结点s
    r->next=s;                             //将s插入r之后
    r=s;
  }
  r->next=NULL;                           //尾结点next域置为NULL
}
```

O(n)

■ 单链表的创建特点：

- ◆ 整个可用存储空间可为多个链表共同享用；
- ◆ 链表占用的空间由系统根据需求即时生成；
- ◆ 该过程是一个动态生成链表的过程。

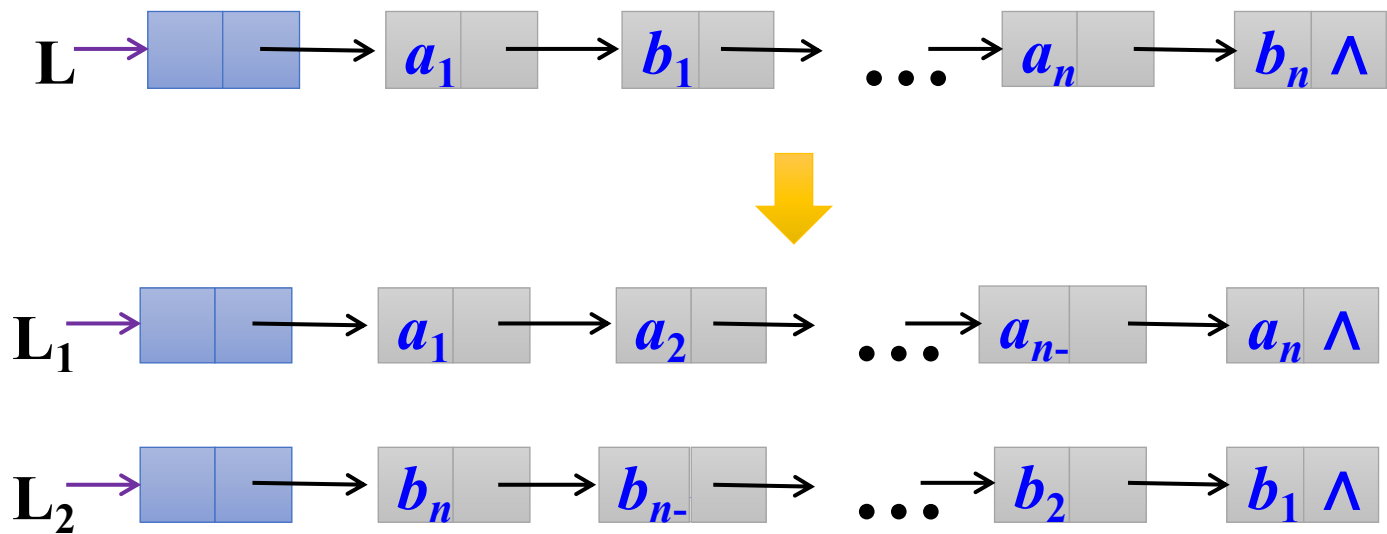
很多算法是以这两个建表算法为基础进行设计的。

单链表的应用示例

【例2.6】 假设有一个带头结点的单链表 $L = (a_1, b_1, a_2, b_2, \dots, a_n, b_n)$ 。设计一个算法将其拆分成两个带头结点的单链表 L_1 和 L_2 :

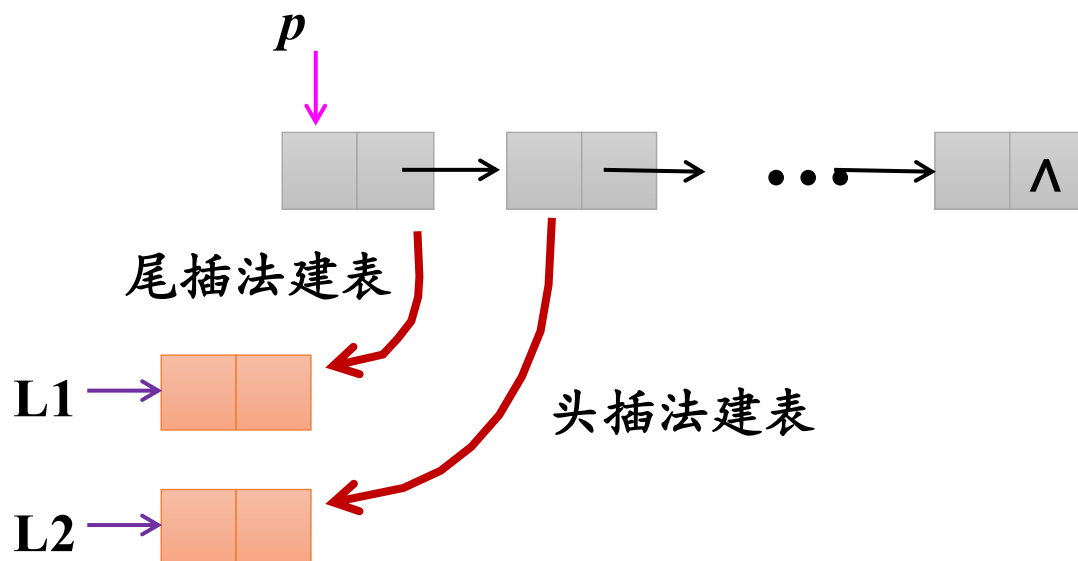
$$L_1 = (a_1, a_2, \dots, a_n), \quad L_2 = (b_n, b_{n-1}, \dots, b_1)$$

要求 L_1 使用 L 的头结点。



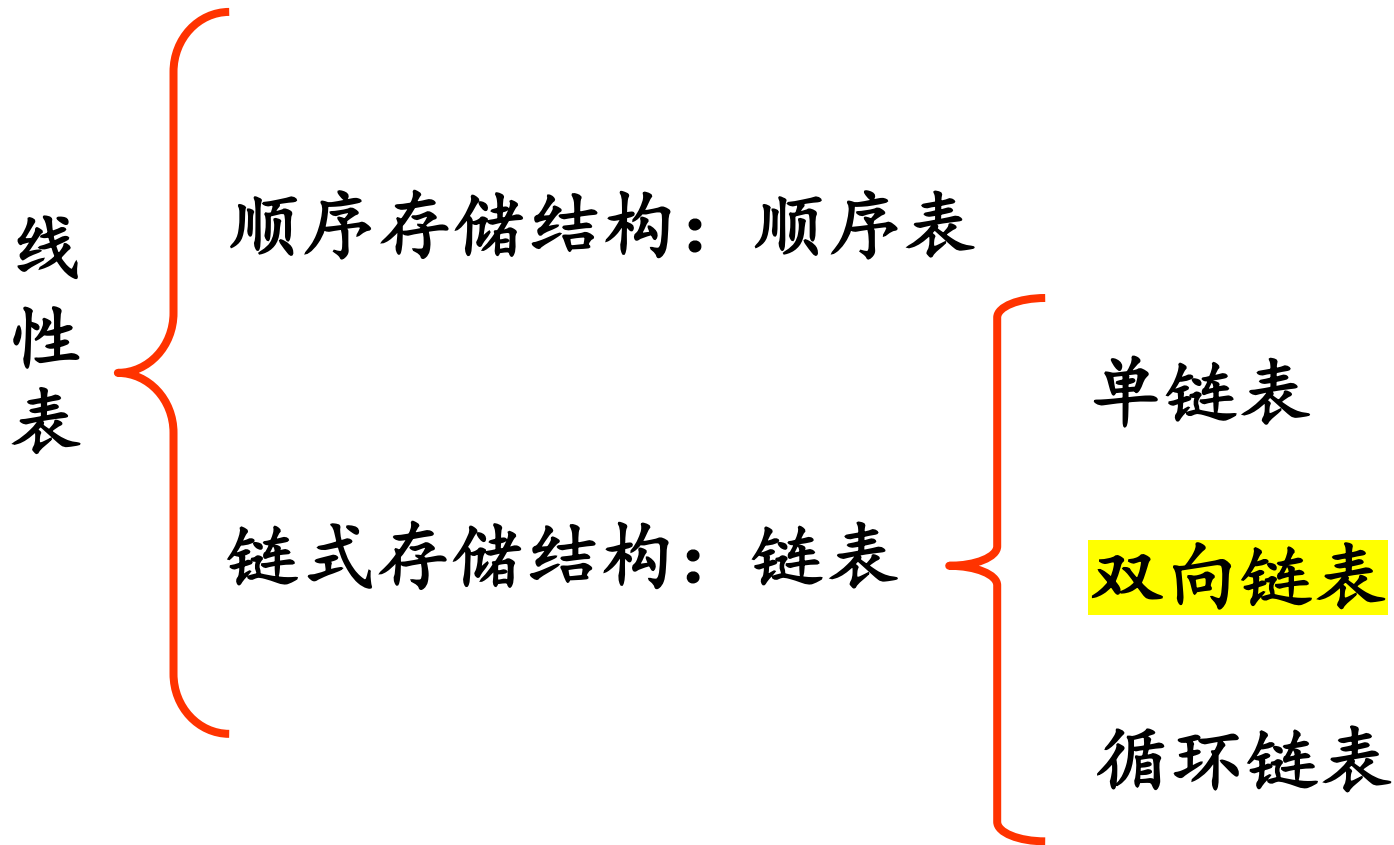
单链表的应用示例

- 利用原单链表L中的所有结点通过改变指针域重组成单链表L1和L2。
- 由于L1中结点的相对顺序与L中的相同，所以采用尾插法建立单链表L1；
- 由于L2中结点的相对顺序与L中的相反，所以采用头插法建立单链表L2。



```
void split(LinkNode *&L, LinkNode *&L1, LinkNode *&L2)
{ LinkNode *p=L->next, *q, *r1; //p指向第1个数据结点
  L1=L; //L1利用原来L的头结点
  r1=L1; //r1始终指向L1的尾结点
  L2=(LinkNode *)malloc(sizeof(LinkNode)); //创建L2的头结点
  L2->next=NULL; //置L2的指针域为NULL
  while (p!=NULL)
  { r1->next=p; //采用尾插法将p(data值为 $a_i$ )插入L1中
    r1=p;
    p=p->next; //p移向下一个结点(data值为 $b_i$ )
    q=p->next; //用q保存p的后继结点
    p->next=L2->next; //采用头插法将p插入L2中
    L2->next=p;
    p=q; //p重新指向 $a_{i+1}$ 的结点
  }
  r1->next=NULL; //尾结点next置空
}
```

线性表 (Linear List) 是数据特性相同的元素构成的有限序列。

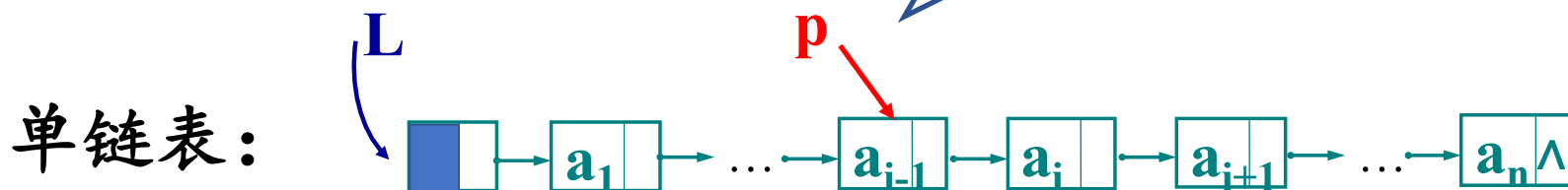


双向链表



可用双向链表来克服单链表的缺点

为什么要讨论双向链表：



单链表的结点→有指示后继的指针域→找后继结点方便

即：查找某结点的后继结点的执行时间为 $O(1)$

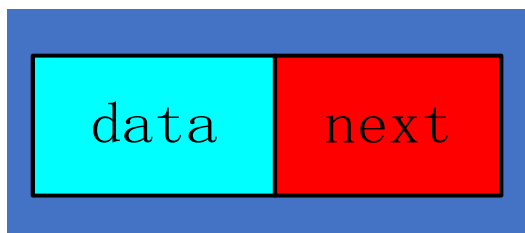
→无指示前驱的指针域→找前驱结点难：从表头出发查找

即：查找某结点的前驱结点的执行时间为 $O(n)$

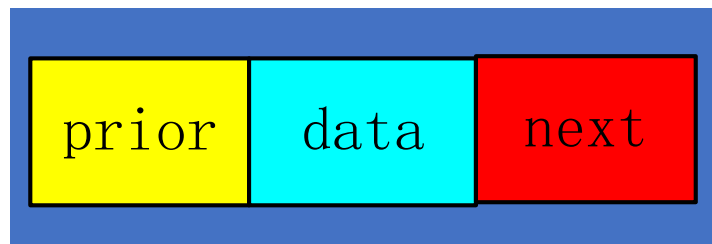
双向链表



在单链表的每个结点里再增加一个指向其前驱的指针域prior，这样链表中就形成了有两个方向不同的链，故称为**双向链表**。



单链表结点

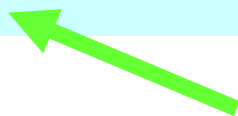


双向链表结点

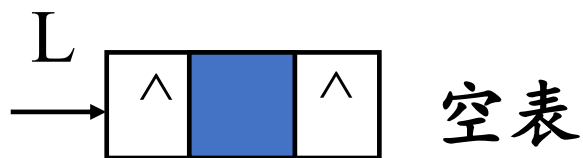
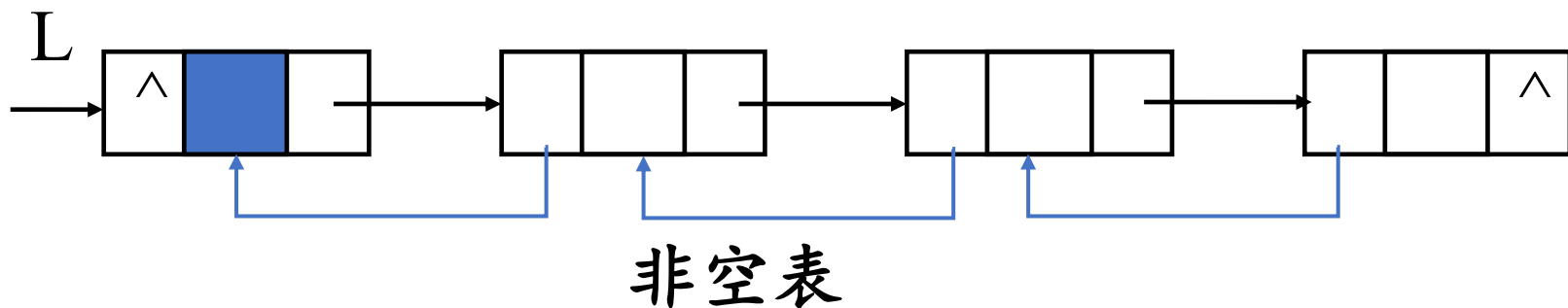
双向链表



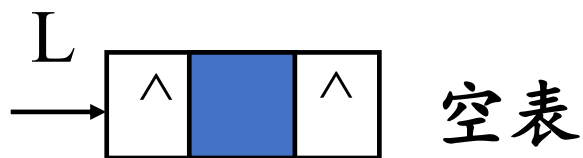
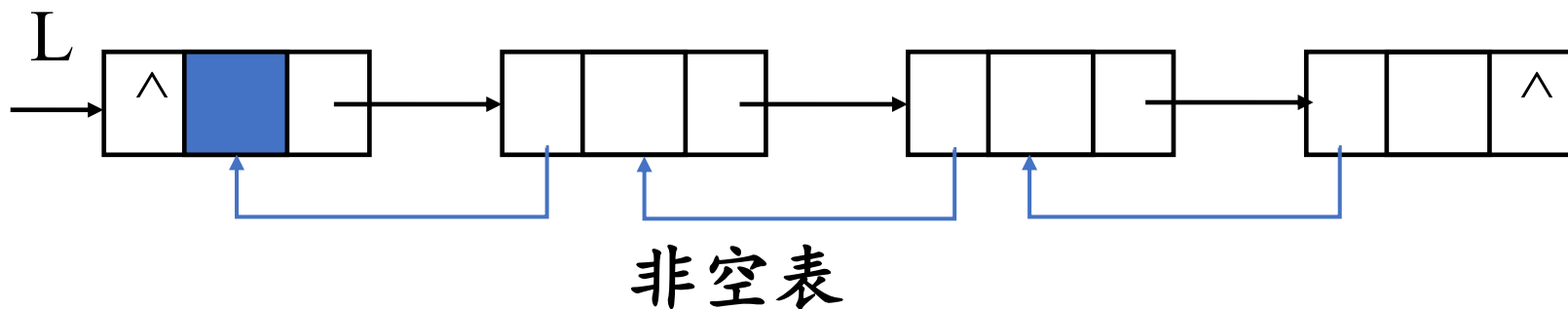
```
typedef struct DNode    //双向链表结点类型
{ ElemType data;
  struct DNode *prior;  //指向前驱结点
  struct DNode *next;   //指向后继结点
} DLinkNode;
```



双向链表结点结构



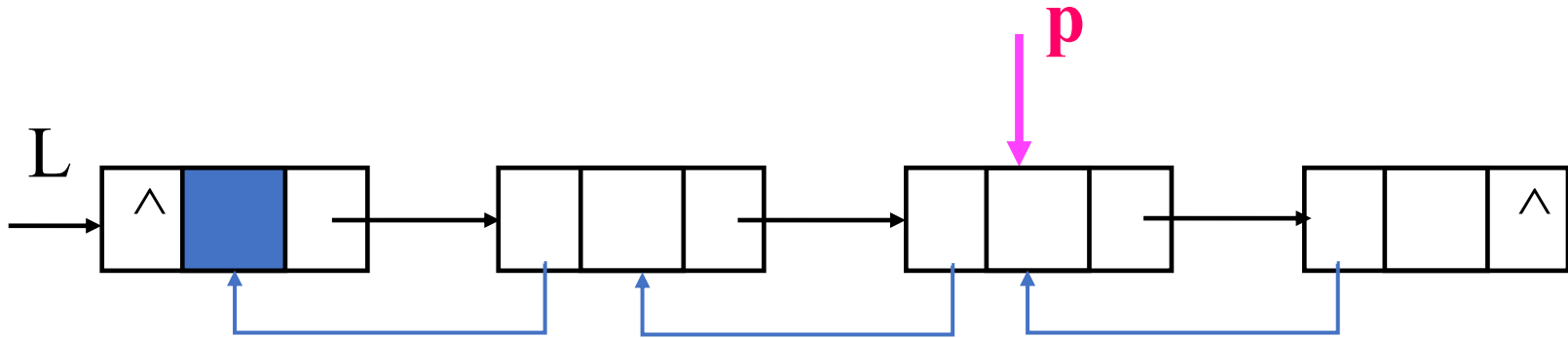
- 非空表：头结点prior域为空，尾结点next域为空
- 空表：头结点prior域为空，头结点 next 域也为空（此时没有尾结点）。



双链表的优点：

- 从任一结点出发可以快速找到其前驱结点和后继结点；
- 从任一结点出发可以访问其他结点。

双向链表结构的对称性（设指针p指向某一结点）

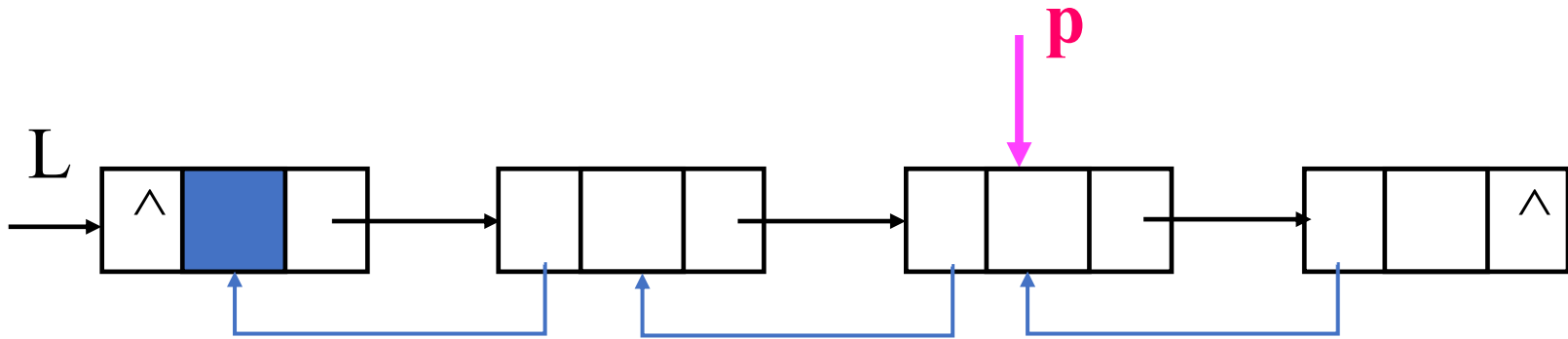


$$p \rightarrow \text{next} \rightarrow \text{prior} = p = p \rightarrow \text{prior} \rightarrow \text{next}$$

双链表的优点：

- 从任一结点出发可以快速找到其前驱结点和后继结点；
- 从任一结点出发可以访问其他结点。

双向链表结构的对称性（设指针p指向某一结点）

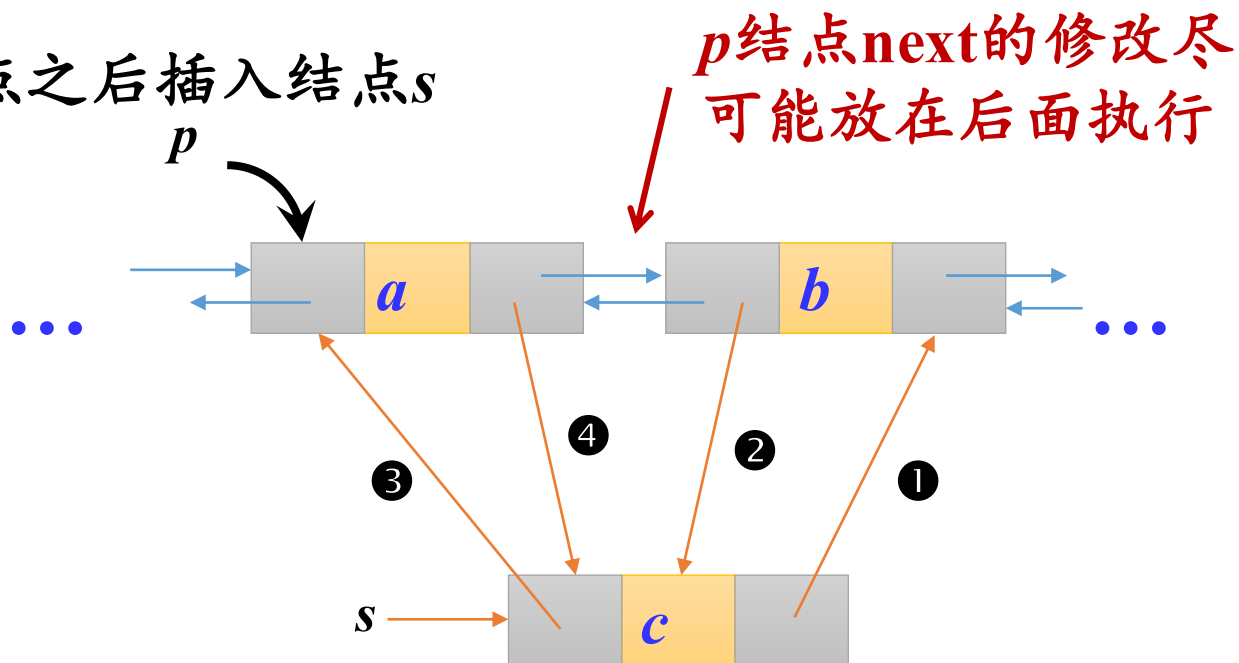


$$p \rightarrow \text{next} \rightarrow \text{prior} = p = p \rightarrow \text{prior} \rightarrow \text{next}$$

双向链表的某些操作（如求表长、查找）和单链表相同；但在插入、删除时，需同时修改两个方向的指针，时间复杂度为 $O(n)$

双向链表的插入

在 p 结点之后插入结点 s



操作语句：

① $s \rightarrow \text{next} = p \rightarrow \text{next}$

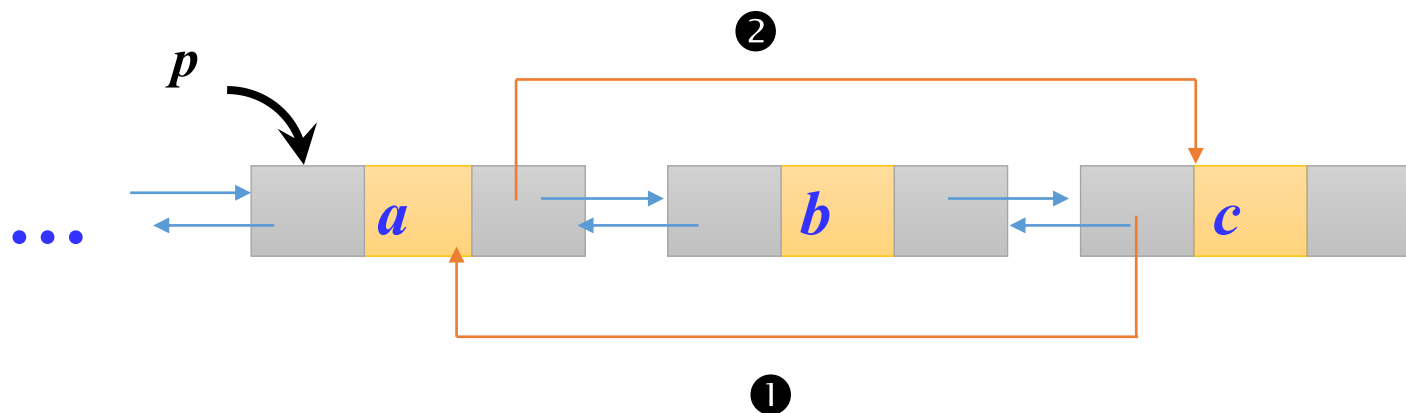
② $p \rightarrow \text{next} \rightarrow \text{prior} = s$

③ $s \rightarrow \text{prior} = p$

④ $p \rightarrow \text{next} = s$ 插入完毕

双向链表的删除

删除 p 结点之后的一个结点



操作语句：

① $p \rightarrow \text{next} \rightarrow \text{next} \rightarrow \text{prior} = p$

② $p \rightarrow \text{next} = p \rightarrow \text{next} \rightarrow \text{next}$

删除完毕

双向链表的建立

整体建立双链表也有两种方法：头插法和尾插法。与单链表的建表算法相似，主要是**插入和删除**的不同。

头插法建立双链表：由含有 n 个元素的数组 a 创建带头结点的双链表 L 。

```
void CreateListF(DLinkNode *&L, ElemType a[], int n)
{ DLinkNode *s; int i;
  L=(DLinkNode *)malloc(sizeof(DLinkNode)); //创建头结点
  L->prior=L->next=NULL;           //前后指针域置为NULL
  for (i=0;i<n;i++)                 //循环建立数据结点
  { s=(DLinkNode *)malloc(sizeof(DLinkNode));
    s->data=a[i];                    //创建数据结点s
    s->next=L->next;                 //将s插入到头结点之后
    if (L->next!=NULL)               //若L存在数据结点，修改前驱指针
      L->next->prior=s;
    L->next=s;
    s->prior=L;
  }
}
```

